

Informe Detallado: Implementación de la Extensión RISC-V Compressed (RVC)

Microarquitectura de Procesadores Pipelined

24 de junio de 2026

Índice

1. Introducción al Concepto de Reducción de 16 bits (RVC)

Para entender cómo se modificó el procesador, primero debemos comprender qué es exactamente la extensión **RVC (Compressed Instructions)** y por qué se inventó.

En la arquitectura base RISC-V de 32 bits (RV32I), absolutamente **todas las instrucciones miden 32 bits (4 bytes)**. Esto facilita enormemente el diseño del procesador porque sabemos que el Program Counter (PC) siempre debe avanzar de 4 en 4 (0x0, 0x4, 0x8...). Sin embargo, los estadísticos descubrieron que en la mayoría de los programas, más del 50 % de las instrucciones realizan operaciones extremadamente simples. Por ejemplo:

- Sumar una constante pequeña a un registro.
- Cargar o guardar datos usando registros populares.
- Hacer saltos relativos cortos.

Para ahorrar memoria y tamaño del código (footprint), RISC-V introdujo la extensión **'C'**. Esta extensión toma las instrucciones más comunes y las comprime.^{en} un formato reducido de **16 bits (2 bytes)**.

1.1. Ejemplo de Compresión

Imagina la instrucción estándar para sumar el número 5 al registro **x8**:

```
addi x8, x8, 5
```

En formato normal de 32 bits (RV32I): Se necesitan 32 bits para representar esta acción. Requiere 5 bits para el registro destino (**rd**), 5 bits para el registro fuente (**rs1**), 12 bits para el número 5, más los códigos de operación (opcode).

```
Bits: [ Inmediato 12-bit ] [ rs1 ] [ funct3 ] [ rd ] [ opcode ]  
      [ 000000000101 ] [ 01000 ] [ 000 ] [ 01000 ] [ 0010011 ] -> 32 bits
```

En formato comprimido de 16 bits (RVC - c.addi): Dado que el registro destino (**x8**) es el mismo que el registro fuente (**x8**), la extensión RVC aprovecha esa redundancia. Solo especifica el registro una vez, asume que la instrucción es una suma, y usa un inmediato más pequeño.

```
Bits: [funct3] [imm[5]] [ rd/rs1 ] [imm[4:0]] [ op ]  
      [ 000 ] [ 0 ] [ 01000 ] [ 00101 ] [ 01 ] -> 16 bits
```

Como puedes ver, los dos últimos bits de la instrucción comprimida son **01**. En RISC-V, **si los dos últimos bits no son 11**, el procesador sabe inmediatamente que está leyendo una instrucción comprimida de 16 bits.

2. El Reto Arquitectónico: El Antes

En nuestro diseño original pipelined de 5 etapas (Fetch, Decode, Execute, Memory, Writeback), la CPU era rígida. Estaba diseñada asumiendo que el universo entero estaba hecho de bloques de 32 bits.

El Código Antes de las Modificaciones (fetch.v):

```
1 // 1. El PC siempre suma 4
2 adder pcadd (.a(PCF), .b(32'd4), .y(PCPlus4F));
3
4 // 2. Se lee siempre una palabra de 32 bits de la memoria
5 imem imem (.a(PCF), .rd(InstrF));
```

Listing 1: Etapa Fetch Original

Este enfoque rígido fallaba catastróficamente al introducir código de 16 bits por dos razones:

1. **El PC se desalinea:** Si ejecutamos una instrucción de 16 bits en la dirección 0x0, la siguiente instrucción comenzará en la dirección 0x2. Sin embargo, nuestra memoria sólo está diseñada para leer múltiplos de 4 (0x0, 0x4, 0x8). Si el PC vale 0x2, al leer la palabra entera, nuestra pequeña instrucción quedará atascada en la parte alta (bits 31 a 16).
2. **Decode espera 32 bits:** La siguiente etapa (Decode) extrae los registros Rs1, Rs2 y Rd de ubicaciones específicas de un arreglo de 32 bits. Si le enviamos una instrucción de 16 bits en bruto, Decode extraerá basura y leerá registros incorrectos.

3. La Solución: Expansión en Tiempo Cero (El Después)

Para no tener que reescribir todo el procesador (lo cual habría destruido la Hazard Unit y complicado inmensamente el Decode), se adoptó una solución sumamente elegante: **ocultarle al pipeline que existen instrucciones de 16 bits**.

La meta fue crear un "traductor" dentro de la primera etapa (Fetch). Cuando llega una instrucción de 16 bits, este traductor la convierte instantáneamente en su equivalente de 32 bits *antes* de que pase a la etapa Decode. Así, Decode siempre ve instrucciones estándar de 32 bits.

A continuación, explicamos cómo se modificó el procesador paso a paso.

3.1. Paso 1: Lectura y Alineación (fetch.v)

Ahora la memoria se lee en bloques de 32 bits estándar, pero introducimos un multiplexor de hardware que revisa si el Program Counter está desalineado.

El bit 1 del PC (PCF[1]) nos indica la alineación a medias palabras. Si el PC vale 0x0 o 0x4, PCF[1] es 0. Si el PC vale 0x2 o 0x6, PCF[1] es 1. Si vale 1, significa que los datos útiles de 16 bits que acabamos de leer de la memoria no están al principio, sino desplazados a la izquierda.

El Código Después:

```

1 wire [31:0] raw_imem_data;
2 imem imem (.a(PCF), .rd(raw_imem_data));
3
4 // Si el PC termina en 2 o 6, la instruccion esta en [31:16]
5 // Recorremos la instruccion hacia la parte baja.
6 wire [31:0] actual_instr;
7 assign actual_instr = PCF[1] ? {16'b0, raw_imem_data[31:16]} :
    raw_imem_data;

```

Listing 2: Alineacion Dinamica

3.2. Paso 2: El Descompresor Combinacional (decompresor.v)

Creamos un módulo nuevo en Verilog. Este módulo extrae los 16 bits más bajos (cinstr), revisa si es una instrucción comprimida, e infla los bits a su versión RV32I.

```

1 // Si termina diferente a 11, esta comprimida
2 assign is_compressed = (cinstr[1:0] != 2'b11);
3
4 always @(*) begin
5     // Traduce un c.addi (16 bits) a un addi normal (32 bits)
6     if (is_compressed && cinstr[15:13] == 3'b000) begin
7         outstr = {imm_addi[11:0], rdp_rslp, 3'b000, rdp_rslp, 7'
            b0010011};
8     end
9 end

```

Listing 3: Traduccion a 32 bits en decompresor.v

Esta conversión ocurre sin retrasos de reloj. Es un circuito combinacional puro.

3.3. Paso 3: Engañando al Pipeline

Ahora que tenemos dos versiones de la instrucción (la original leída de memoria, y la descomprimida generada por el traductor), usamos un multiplexor guiado por la bandera `is_compressed` para elegir cuál enviar a la etapa Decode a través del registro IF/ID.

```

1 // Si la instruccion era de 16 bits, entregamos el resultado del
    traductor.
2 // Si era de 32 bits, entregamos la lectura normal de memoria.
3 assign InstrF = is_compressed ? decompressed_instr : actual_instr;

```

Listing 4: El Engano Final

3.4. Paso 4: Actualización Dinámica del PC

Por último, debemos decirle al Program Counter cuánto tiene que avanzar. Ya no podemos avanzar +4 rígidamente. Si la instrucción que acabamos de leer fue de 16 bits, el PC sólo debe avanzar +2 bytes para apuntar correctamente a la instrucción que sigue inmediatamente después.

```

1 wire [31:0] pc_increment;
2
3 // Multiplexor de incremento: +2 o +4

```

```

4 assign pc_increment = is_compressed ? 32'd2 : 32'd4;
5
6 adder pcadd (.a(PCF), .b(pc_increment), .y(PCPlus4F));

```

Listing 5: Calculo dinamico del PC

4. Impacto en el Resto de la Arquitectura

Gracias a haber solucionado el problema desde el **Fetch**, los demás subsistemas del CPU funcionaron sin apenas tocarlos.

4.1. La Unidad de Riesgos (Hazard Unit)

El Antes: La `hunit.v` comparaba las direcciones de los registros (bits del 15 al 19 para `Rs1`, etc.) para detectar si una instrucción en **Execute** necesitaba enviar datos adelantados (Forwarding) a una instrucción en **Decode**, o si debía insertar burbujas (Stall) por un **Load**.

El Después: No se modificó **ni una sola línea de código** en `hunit.v`. ¿Cómo es posible? Dado que el traductor (descompresor) se ubicó *antes* del registro que divide las etapas **IF** y **ID**, cuando llega el flanco positivo del reloj, la etapa **Decode** siempre recibe un bloque de 32 bits perfecto. **Decode** extrae `Rs1`, `Rs2` y `Rd` de los lugares estándar, la Hazard Unit los lee y aplica toda su lógica de prevención de riesgos de la misma forma que lo hacía antes. La Hazard Unit no tiene idea de que en el código fuente existía una instrucción comprimida.

4.2. Etapas de Execute, Memory y Writeback

Funcionan igual. La ALU sigue sumando registros de 32 bits. La **Data Memory** sigue leyendo palabras completas. Lo único diferente ocurre si usamos instrucciones como `c.lw` (Load Word Comprimido), pero de nuevo, como el descompresor lo transformó a un `lw` normal, las señales de control `MemWrite` o `ResultSrc` se encienden sin problema.

5. Conclusión

La compresión de 16 bits en RISC-V es un ejemplo brillante de eficiencia arquitectónica. Al implementar la expansión en tiempo cero directamente al leer la memoria, logramos un procesador retro-compatible que puede intercalar sin penalización instrucciones **RV32I** e instrucciones **RVC** en un mismo programa, manteniendo la elegancia y limpieza del diseño Pipelined de 5 etapas.